KALPANA REDDY / LEARNING NOTES

RAG, Knowledge Graphs & AI Retrieval Systems

A practical beginner-to-intermediate guide with examples, architectures, and selection advice.

Concepts • Comparisons • Practical examples • Learning roadmap

A practical beginner-to-intermediate guide with examples, architectures, and selection advice

What you will learn	Practical outcome
Core concepts: LLMs, tokens, context windows, embeddings, retrieval	Understand the vocabulary and how the pieces fit together
RAG, vector databases, knowledge graphs, Neo4j, GraphRAG	Know what each technology does and when it is useful
Common combinations and architectures	Choose a sensible design for a real application
Confluence + Salesforce examples	Apply the ideas to a realistic company knowledge assistant
Evaluation, security, cost, and a learning roadmap	Know what to build and test next

Prepared as a learning guide

How to use this guide

Read in order the first time. The concepts build on each other. You do not need to learn Neo4j, vector databases, and every framework at once. Start with the problem you want the AI to solve, then choose the simplest retrieval approach that works.

Part	Focus
1–2	LLMs, tokens, context, and why retrieval is needed
3–6	RAG, chunking, embeddings, vector search, and reranking
7–9	Knowledge graphs, Neo4j, GraphRAG, and combinations
10–12	Architectures, practical examples, implementation choices
13–15	Evaluation, security, glossary, and learning plan

1. The big picture: what problem are we solving?

Imagine a company has 128 Confluence pages describing systems, APIs, policies, incidents, owners, and business processes. Together they contain about 1.3 million tokens. A user asks: “Which service owns customer sync, and what breaks if it is unavailable?”

An LLM is good at understanding and writing language, but it may not know the company’s private or latest information. We need a way to find the right internal information and provide it to the model at answer time. This is the central motivation for retrieval-augmented generation (RAG).

Component	Plain-English meaning	Example
LLM	The language model that reasons over input and generates text	GPT, Claude, Gemini, or another model
Source data	The company information the answer should be based on	Confluence pages, PDFs, Salesforce, BigQuery
Retrieval	Finding information relevant to a question	Find the page about customer sync
Context	The information included in the model’s prompt	Two relevant paragraphs and their page links
Generation	The model’s response based on the question and context	Explain the owner and dependencies

2. Essential basics before RAG

2.1 What is an LLM?

A Large Language Model (LLM) is an AI model trained to process and generate language. It can summarize, explain, classify, write code, and answer questions. It does not automatically have access to your private Confluence or Salesforce data. An application must provide that data or connect the model to tools.

2.2 What is a token?

A token is a unit of text used by a model. It can be a word, part of a word, punctuation, or another text unit. Token counts are not the same as word counts. The reported 1.3 million tokens describes the approximate size of the content after tokenization.

2.3 What is a context window?

The context window is the amount of tokenized input and output a model can handle in a single request. The exact limit depends on the model. The prompt, retrieved content, conversation history, and answer all use tokens. Even if a model can technically accept a huge context, sending every page for every question may cost more, take longer, and make it harder to focus on the most relevant evidence.

2.4 What is a prompt?

A prompt is the instruction and information given to the model. For example: “Answer only from the context below. If the answer is not supported, say so. Include source links.” RAG supplies relevant source content as part of that prompt.

2.5 What is an embedding?

An embedding is a list of numbers representing text in a way that captures aspects of its meaning. Texts with similar meaning often have nearby vectors in the embedding space. An embedding model creates these vectors; it is not the same thing as the LLM that writes the final answer.

Text: “How do I rotate an API credential?”
Possible semantically similar text: “Steps for renewing a service token.”

An embedding-based search can match these even when they do not share the same keywords. Embeddings are useful, but they do not guarantee that two passages are factually equivalent.

3. What is RAG?

RAG stands for Retrieval-Augmented Generation. It is an application pattern: retrieve relevant information from an external source, augment the prompt with that information, and ask the LLM to generate an answer grounded in it.

3.1 The two stages of RAG

Stage	What happens	Typical work
A. Prepare / index data (mostly offline)	Make the source content searchable	Extract pages, clean content, chunk text, create embeddings, store chunks and metadata
B. Answer a question (online)	Retrieve evidence and generate a response	Interpret question, search, select/rerank results, build prompt, call LLM, return answer and citations

3.2 A simple RAG example

1. A Confluence connector reads the pages and keeps the page title, URL, space, owner, and last-updated time.
2. The content is split into manageable chunks, often by headings or paragraphs.
3. An embedding model creates a vector for each chunk; the vectors and original text are indexed.
4. A user asks: “How does the customer sync retry failures?” The system searches for relevant chunks.
5. The best chunks, along with source metadata, are included in the LLM prompt.
6. The LLM answers using those sources and can cite the Confluence pages.

3.3 What RAG is—and is not

- RAG is not a specific database or model. It is a pattern that can use different search and storage technologies.
- RAG does not train or fine-tune the LLM. It supplies information at inference time.
- RAG does not guarantee correctness. Retrieval can miss evidence, content can be stale, and the model can misinterpret context.
- RAG is not always vector search. Retrieval can use keyword search, SQL/SOQL, graph traversal, metadata filters, or a hybrid of these.

3.4 When to use RAG

- The answer depends on private, internal, changing, or domain-specific documents.
- You want answers grounded in documents with source links.
- The corpus is too large or changes too often to paste into every prompt.
- You need to control which content each user is allowed to retrieve.

4. Chunking and indexing

Chunking means splitting long content into smaller passages. Search typically works better on focused passages than on one giant page. Chunk size and overlap are design choices, not universal constants.

Choice	Benefit	Risk / trade-off
Small chunks	Precise retrieval and less irrelevant text	May lose context such as definitions or qualifications
Large chunks	More surrounding context	Can be noisy and use more prompt tokens
Overlap between chunks	Preserves information near split boundaries	Duplicates text and increases index size
Split by headings / sections	Keeps topic structure and titles	Requires reasonable source formatting

Useful metadata includes source URL, page title, section heading, space, updated timestamp, document type, access-control information, and chunk position. Metadata helps filtering, citations, freshness checks, and permissions.

Indexing is the process of preparing data for search. For a Confluence corpus, indexing may include extraction, cleanup, chunking, embedding generation, and storing the text and metadata.

5. Vector databases and semantic search

A vector database stores and searches vectors, often alongside the original text and metadata. A query is embedded into a vector; the system finds vectors that are similar to it and returns the corresponding text chunks.

5.1 Why use one?

- Semantic matching: find passages by meaning, not only exact keywords.
- Scale: organize and search many chunks efficiently.
- Metadata filtering: narrow search to a product, team, space, or date when supported.
- Integration: connect retrieval results to a RAG pipeline.

5.2 Common options

Option	What it is	Useful when
Pinecone	Managed vector database service	You want a hosted vector- search service
Weaviate	Vector database with search and data features	You want vector and hybrid search capabilities
Chroma	Vector store commonly used in prototypes and applications	You want to learn or build a smaller prototype
PostgreSQL + pgvector	Relational database with vector search support	You already use PostgreSQL and want fewer separate systems
OpenSearch / Elasticsearch	Search engines with keyword and vector capabilities	Keyword, filters, and semantic search need to coexist
Neo4j vector index	Vector search supported within a graph database	You want vectors and graph relationships in one system

These products differ in deployment, scale, operations, integrations, cost, and features. The table is a conceptual guide, not a performance ranking.

5.3 Vector search is not the same as a vector database

Vector search is the retrieval technique. A vector database is one possible system for storing vectors and performing that search. Some relational databases and search engines also support vector search.

6. Keyword search, hybrid search, and reranking

6.1 Keyword search

Keyword search finds exact terms, phrases, identifiers, and lexical matches. It is often excellent for error codes, API names, field names, ticket IDs, and exact policy wording.

6.2 Hybrid search

Hybrid search combines keyword retrieval and vector/semantic retrieval. This can work well for technical documentation because users may ask conceptually while the answer contains exact names or identifiers.

Example question: “Why does POCAccountResource fail with error 401?”
Keyword search helps find the exact class and error.
Vector search helps find semantically related authentication troubleshooting.
Hybrid search combines the candidate results.

6.3 Reranking

A reranker takes a set of candidate passages and scores which ones best answer the specific question. Retrieval might find 30 candidates; reranking can prioritize the 5–10 most useful. This adds processing and possibly latency, but may improve relevance.

Technique	Best at	Limitation
Keyword	Exact terms and identifiers	Can miss synonyms and paraphrases
Vector / semantic	Conceptual similarity and paraphrases	Can miss exact identifiers or return related-but-not-answering text
Hybrid	Combining exact and semantic matches	Needs tuning and deduplication
Reranking	Ordering candidates for a specific question	Extra model or compute step

7. Knowledge graphs

A knowledge graph represents entities and the relationships between them. Entities are commonly represented as nodes; relationships are edges. Properties can describe each node or relationship.

```
(:Service {name: "Customer Sync"})
-[:OWNED_BY]-> (:Team {name: "CRM Platform"})
-[:DEPENDS_ON]-> (:Service {name: "Identity API"})
```

This structure explicitly records that Customer Sync is owned by CRM Platform and depends on Identity API. A graph can be traversed to find connected entities and paths.

7.1 When a knowledge graph helps

- Questions depend on relationships across multiple pages or systems.
- You need to trace ownership, dependencies, impact, lineage, or multi-hop connections.
- The same entities appear repeatedly under different names and need to be linked consistently.
- You need to explain how entities connect, not only find a passage that mentions them.

7.2 When a graph may be unnecessary

- The application mostly answers questions directly from a paragraph or policy.
- There are few meaningful relationships to traverse.
- A well-built keyword/vector search already meets quality needs.
- The cost of extracting, validating, and maintaining graph relationships exceeds the benefit.

8. Neo4j: what it is

Neo4j is a graph database. It stores nodes, relationships, and properties, and supports graph queries using a language called Cypher. Neo4j is not itself an LLM and is not the same thing as RAG.

8.1 A practical example

Suppose Confluence pages mention a customer-sync service, a Salesforce integration, a service owner, a downstream API, and an incident. A graph model could connect these entities and let the application trace the dependency chain.

Graph element	Example
Node / entity	Service: Customer Sync
Node / entity	Team: CRM Platform
Relationship	Customer Sync — OWNED_BY → CRM Platform
Relationship	Customer Sync — DEPENDS_ON → Identity API
Property	Service status, URL, last verified date, source page

8.2 What Neo4j can contribute to an AI app

- Store a knowledge graph and query connected entities.
- Find multi-hop relationships such as service → API → team → incident.
- Support graph-based retrieval and, in supported configurations, vector similarity search.
- Return evidence or subgraphs that an LLM can use to explain an answer.

Graph data must be created and maintained. It may be extracted from text using an LLM, imported from structured systems, or assembled through a combination. Extracted relationships can be wrong, so validation and source traceability matter.

9. GraphRAG

GraphRAG refers broadly to approaches that combine retrieval-augmented generation with graph structures. The term is used for more than one implementation. Some systems construct a knowledge graph from a document collection and use graph structure, communities, or summaries for retrieval; others use an existing graph to retrieve relevant connected facts.

9.1 How a GraphRAG pipeline can work

1. Extract text from Confluence pages.
2. Identify entities and relationships, such as services, teams, APIs, and dependencies.
3. Create a graph and retain links back to the source passages.
4. At question time, identify relevant entities or graph regions and retrieve connected facts and supporting text.
5. Give the evidence to an LLM to synthesize an answer with source references.

9.2 GraphRAG versus ordinary RAG

Question type	Likely useful approach
“What does the deployment policy say?”	Standard RAG over relevant document chunks
“Which team owns Service X?”	Keyword/vector RAG may be enough; graph lookup helps if ownership is modeled
“Which services could be affected if Identity API fails?”	Graph retrieval can be especially useful
“Summarize the recurring themes across all engineering pages.”	A graph/community-summary approach may help; global summarization is a different retrieval challenge

GraphRAG is not automatically better than standard RAG. Graph construction can be expensive, extraction may introduce errors, and graphs require updates when source documents change. Test it against a simpler baseline.

10. Important related terms

Term	Meaning	Role in a solution
Retriever	Component that finds candidate evidence	Searches documents, vectors, graphs, or databases

Term	Meaning	Role in a solution
Embedding model	Model that maps text to vectors	Enables semantic similarity search
Reranker	Model or algorithm that reorders candidates	Improves relevance of retrieved evidence
Knowledge graph	Entities linked by explicit relationships	Represents structure and connections
Graph database	Database designed to store/query graph structures	Neo4j is an example
GraphRAG	RAG using graph structure in retrieval and/or summarization	Useful for connected or multi-hop questions
Long-context LLM	LLM that accepts a large input context	Can read more text in one request, within its limit
Summarization / compression	Creates a shorter representation of source material	Reduces prompt size; may omit details
SQL / SOQL retrieval	Queries structured tables or Salesforce objects	Best for exact current fields, filters, and aggregates
Agent / tool calling	LLM-driven selection and use of tools	Can query APIs, databases, search, or take actions
Fine-tuning	Further training a model on examples	Changes behavior or task performance; not the default way to inject current documents
MCP	A protocol for connecting AI clients to tools and data sources	Provides access to capabilities; does not itself mean RAG

11. Common architectures and combinations

These terms are not mutually exclusive. A real application can combine retrieval methods. Choose based on the questions, source data, quality requirements, freshness, permissions, latency, and operational budget.

11.1 Standard RAG + vector database

```
Confluence → extract/clean → chunk → embed → vector index
User question → embed/search → relevant chunks → LLM → answer + citations
```

Use when the main job is finding relevant passages in a document collection. This is a sensible baseline for the 128-page Confluence example.

11.2 Neo4j + GraphRAG

```
Confluence → extract entities/relationships → Neo4j graph
Question → graph retrieval/traversal → connected facts + source text → LLM
```

Use when answers depend on relationships, dependencies, ownership, or multi-hop reasoning. GraphRAG can also include summaries and text retrieval; the exact design varies.

11.3 Graph + vector search (hybrid GraphRAG)

```
Confluence → chunks + embeddings → vector index
↘ entities + relationships → Neo4j
Question → semantic retrieval + graph traversal → merge evidence → LLM
```

Use when you need both passage-level semantic search and relationship traversal. This can be powerful but adds architecture and maintenance complexity.

11.4 Hybrid keyword + vector RAG + reranking

```
Question → keyword search + vector search → merge/deduplicate → rerank → LLM
```

Use for technical knowledge bases with exact identifiers, error codes, and natural- language questions. This is often a strong upgrade before introducing a graph.

11.5 SQL/SOQL + RAG

```
Structured question → SOQL/SQL for current facts
Document question → RAG for policies, design docs, and explanations
Combine results → LLM → answer
```

Use when a system needs both current structured facts and explanations from documents. For example, retrieve an opportunity's current stage from Salesforce and the relevant sales process from Confluence.

11.6 Long context or summarization without a retrieval index

```
All pages → long-context model, OR pages → summaries → combined context → LLM
```

Use for one-off analysis, smaller corpora, or when the task genuinely needs broad coverage of the whole corpus. Repeatedly sending all source content may be expensive; summaries can lose details and should link back to original evidence.

11.7 Agent + retrieval tools

```
User question → agent decides which tool to use
→ Confluence search / vector search / Neo4j / Salesforce SOQL / BigQuery
→ gather results → LLM synthesizes response
```

Use when questions require multiple steps or different data sources. An agent is an orchestration pattern; it can use RAG, graphs, and structured queries rather than replacing them.

12. Worked example: 128 Confluence pages / 1.3 million tokens

Goal: Build an assistant that answers questions about internal architecture and processes, with evidence from Confluence.

12.1 Recommended first version

1. Extract page content through the Confluence API and retain page IDs, URLs, titles, headings, update timestamps, and access permissions.
2. Clean the text and split it into meaningful chunks using headings and paragraph boundaries. Tune chunk size on a test set rather than choosing a universal number.
3. Index the chunks using keyword search and/or embeddings. A vector-capable search system is one option; it is not mandatory to use a dedicated vector database.
4. At query time, apply permission filters, retrieve candidate passages, optionally rerank them, and select a limited set of evidence.
5. Prompt the LLM to answer from the evidence, acknowledge gaps, and cite page URLs and titles.
6. Evaluate with real questions and expected sources; monitor stale pages, retrieval misses, access-control leaks, latency, and cost.

12.2 Add Neo4j only if the questions justify it

If users frequently ask “what depends on this?”, “which team owns these connected services?”, or “what is the impact across systems?”, build a graph of services, teams, APIs, incidents, and dependencies. Preserve links from graph facts to source passages. Compare graph-assisted retrieval with the standard RAG baseline.

12.3 Example answer flow

User asks	Retrieval plan	Why
“What is the retry policy?”	Keyword/vector retrieval over policy pages	The answer likely exists in a few passages
“Which API owns this field?”	Search docs; possibly query Salesforce metadata or a graph	May require an exact identifier and ownership relationship
“What breaks if Identity API is down?”	Graph traversal plus source passages	Requires connected dependency paths
“What changed in the policy last month?”	Metadata/date filters and version/change history	Freshness and change records matter

User asks	Retrieval plan	Why
“What is the current opportunity amount?”	Salesforce SOQL or trusted data API	Current structured data should be queried directly

13. Choosing the right approach

Need	Start with	Consider adding
Q&A over company documents	Standard RAG with keyword/vector retrieval	Hybrid search, reranking, citations
Exact technical names and error messages	Keyword + vector hybrid search	Reranking and metadata filters
Relationships and dependency impact	Knowledge graph / graph retrieval	Neo4j + GraphRAG
Current CRM records and metrics	SOQL/SQL or a trusted API	RAG for related explanatory documents
One-time review of a manageable corpus	Long-context model or summarization	RAG if repeated questions or traceability are needed
Multi-step question across many tools	A controlled tool-calling workflow	Agent orchestration with RAG/SQL/graph tools

Rule of thumb: begin with the simplest design that can pass your evaluation set. Add a vector index, graph, reranker, or agent only when it solves a demonstrated failure or requirement.

14. Evaluation, security, and production considerations

14.1 Evaluate more than the final answer

- Retrieval quality: did the system retrieve the pages that actually contain the answer?
- Grounding: is each important claim supported by retrieved evidence?
- Citation accuracy: do the links point to the correct source and relevant section?
- Completeness: did the answer include important caveats or related dependencies?
- Freshness: does the index reflect changed or deleted Confluence pages?
- Latency and cost: how many retrieval calls, model calls, and input tokens are required?
- Permission safety: can a user retrieve content they are not authorized to see?

14.2 Security matters

A RAG index must preserve source permissions. Do not assume that because a service account can read every Confluence page, every end user should be able to search every page. Enforce access controls at retrieval time and protect logs, embeddings, cached results, and source text. Treat retrieved document content as data, not as trusted instructions; a page can contain malicious or irrelevant prompt-like text.

14.3 Keep the index current

Use incremental synchronization where possible. Track page IDs, versions, timestamps, and deletions. Reprocess changed content and remove stale chunks. For graph systems, update or remove relationships when source evidence changes. For every extracted graph fact, retain provenance so it can be checked.

14.4 Avoid common mistakes

- Sending every document to the LLM for every question without checking cost or need.
- Assuming vector similarity means the passage answers the question.
- Adding Neo4j or an agent before proving a simple retrieval baseline is insufficient.
- Creating chunks without retaining source URLs, headings, permissions, or timestamps.
- Treating an LLM-extracted graph relationship as certainly true without validation.
- Measuring only whether the answer sounds good instead of checking evidence and citations.

15. Learning roadmap

Step	Learn / build	Success criterion
1. Foundations	LLM, tokens, context window, prompt, embeddings	Explain the difference without mixing up model and database
2. Basic retrieval	Index a few pages and run keyword search	Find the right source for a question
3. Vector RAG	Chunk, embed, store, retrieve, generate	Answer questions with useful citations
4. Improve retrieval	Hybrid search, metadata filters, reranking	Reduce missed evidence and irrelevant chunks
5. Measure quality	Create 20–50 representative questions with expected sources	Compare retrieval and answer quality before/after changes
6. Knowledge graphs	Model entities and relationships; learn basic Cypher if using Neo4j	Answer a multi-hop dependency question
7. GraphRAG	Combine graph retrieval with source passages	Show measurable improvement on graph-shaped questions

Step	Learn / build	Success criterion
8. Production	Permissions, sync, monitoring, latency, cost, evaluations	Operate safely and detect regressions

Suggested hands-on mini-project

Build a Confluence Q&A prototype in phases. Start with 5–10 pages, then scale to all 128. Create a small evaluation set of real questions and source pages. Compare (A) keyword search, (B) vector RAG, (C) hybrid search with reranking, and only then (D) graph-assisted retrieval for questions involving dependencies. Record answer quality, citation correctness, latency, and cost for each approach.

16. Glossary

Term	Short definition
RAG	Retrieve external information, add it to the prompt, and generate an answer.
LLM	Language model that processes and generates text.
Token	Unit used to represent text inside a model.
Context window	Maximum input/output token capacity for a model request.
Embedding	Vector representation of text or another item.
Vector database	System for storing and searching vectors, often with text and metadata.
Chunk	A smaller piece of a source document.
Retriever	Component that finds candidate evidence.
Reranker	Component that reorders candidate evidence by relevance.
Hybrid search	Combination of keyword and semantic/vector search.
Knowledge graph	Entities connected by explicit relationships.
Neo4j	Graph database with nodes, relationships, properties, and Cypher queries.
GraphRAG	RAG approach that uses graph structures for retrieval and/or summaries.
Provenance	Information showing where a fact came from.

Term	Short definition
Grounding	Basing an answer on evidence rather than unsupported claims.
Fine-tuning	Further training a model to improve or change behavior.
Tool calling	A model requests that an application execute a defined tool or API.
Agent	An application pattern where a model selects steps/tools to pursue a task.
MCP	A protocol for connecting AI clients to tools and data sources.
SOQL / SQL	Query languages for structured Salesforce / relational data.

Final takeaways

- RAG is the overall pattern, not one product.
- A vector database helps find semantically similar text; it does not itself write the answer.
- Neo4j stores and queries relationships; GraphRAG uses graph structure to improve retrieval and reasoning over connected information.
- Standard RAG + vector or hybrid search is a sensible starting point for Confluence document Q&A.
- Combine vector retrieval, graphs, SQL/SOQL, summarization, and agents only when the question types and measured results justify them.
- For the 128-page / 1.3-million-token case, the token count alone cannot reveal which architecture was used. Look for clues such as “embeddings/top-k,” “nodes/edges/Cypher,” or “summaries/context compression.”